

Artificial Intelligence

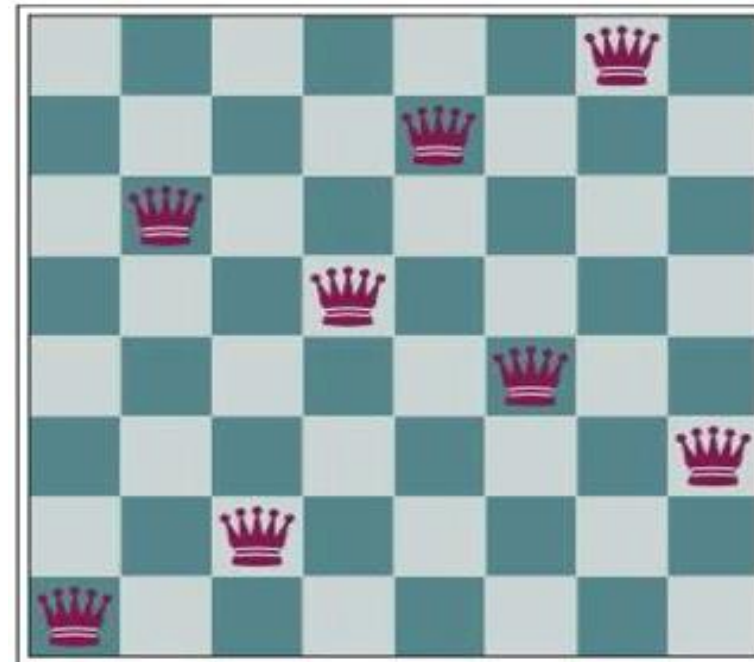
Section 8

By: Mohamed Fathy

Section Content

- Search in Complex Environments
- Hill-Climbing Search

- The previous algorithms addressed problems in **fully observable, deterministic, static, known environments** where the solution is a sequence of actions. Now, we relax those constraints.
- The search algorithms that we have seen so far are designed to explore search spaces **systematically. This systematicity is achieved by keeping one or more paths in memory and by recording which alternatives have been explored at each point along the path.** When a goal is found, the path to that goal also constitutes a solution to the problem.
- In many problems, however, the path to the goal is **irrelevant**. For example, in the 8-queens problem, what matters is the final configuration of queens, not the order in which they are added.

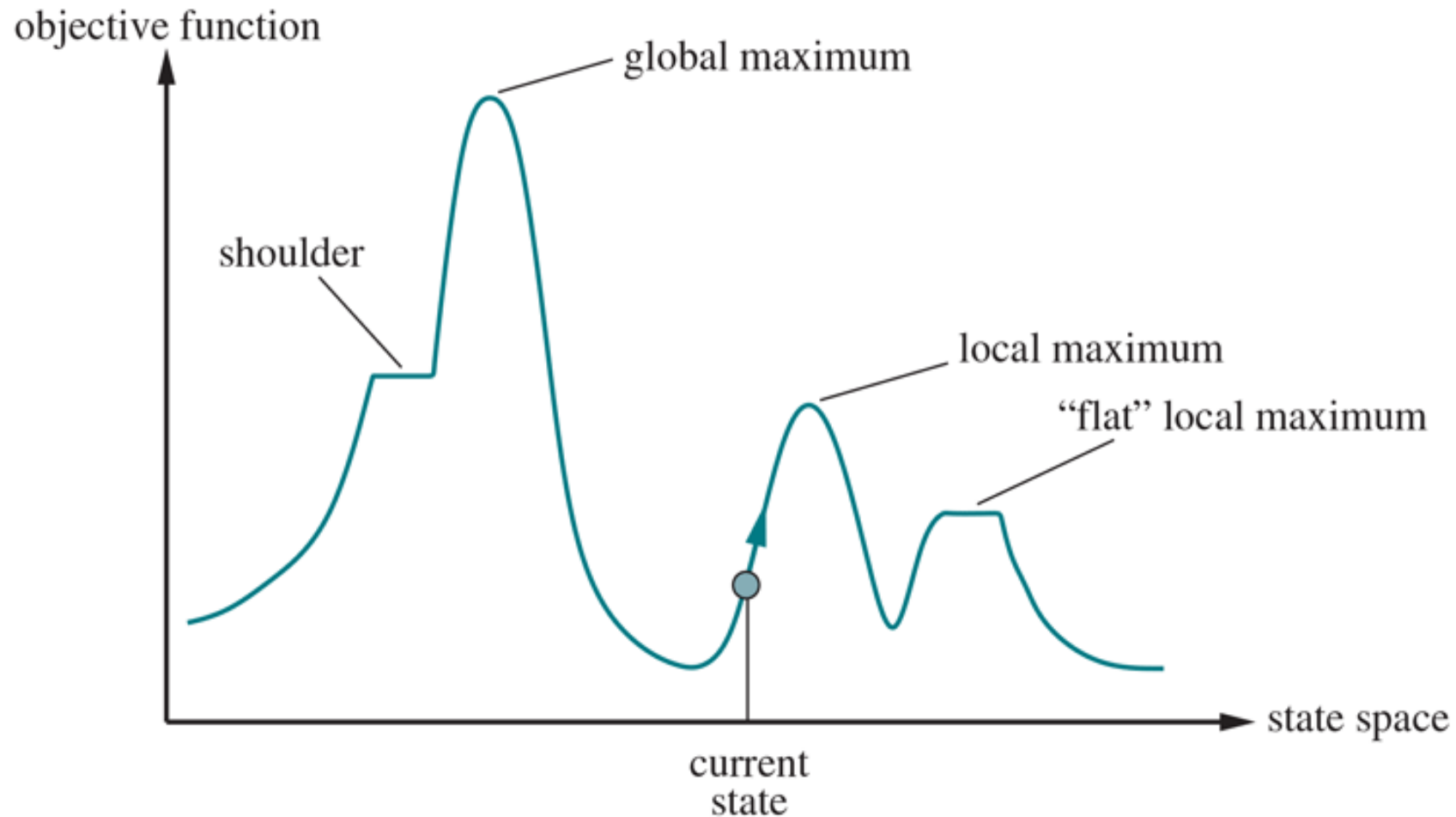


Search in Complex Environments

Hill-Climbing
Search

- **Local search algorithms** operate by searching from a start state to neighboring states, **without keeping track of the paths, nor the set of states that have been reached**. That means they are not systematic—they might never explore a portion of the search space where a solution actually resides.
- However, they have two key advantages:
 1. **They use very little memory.**
 2. **They can often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable.**
- In addition to finding goals, local search algorithms are useful for solving **pure optimization problems**, in which the aim is to find the best state according to an **objective function**.

- To understand local search, we find it useful to consider the **state-space landscape**.
- A **complete local search** algorithm always finds a goal if one exists; an **optimal algorithm** always finds a global minimum/maximum.

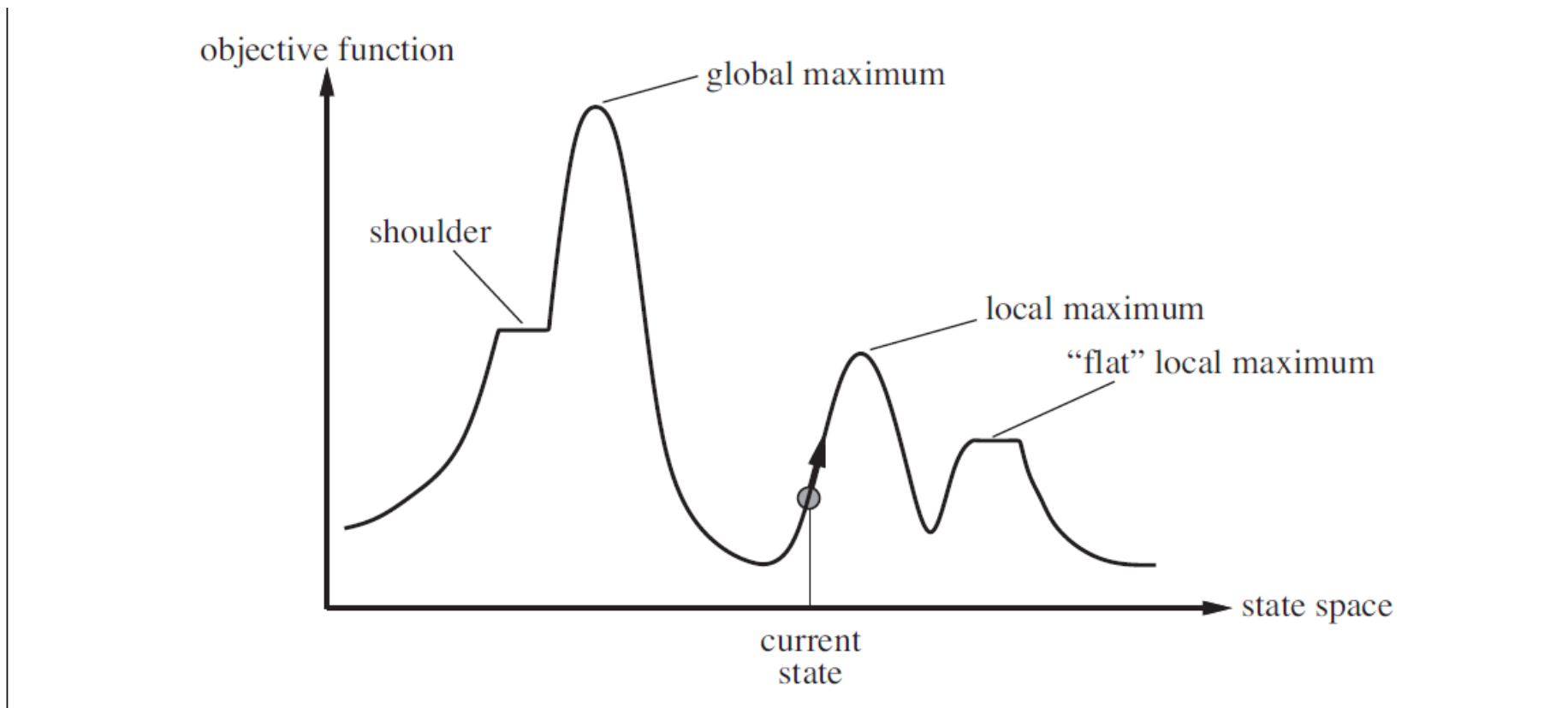


Search in
Complex
Environments

Hill-Climbing
Search

- The **hill-climbing search** algorithm (**steepest-ascent version**) is simply a loop that continually moves in the direction of increasing that is, uphill. It terminates when it reaches a where no neighbor has a higher value. Hill climbing does not look ahead beyond the immediate neighbors of the current state.
- The algorithm does not maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function.

Search in
Complex
Environments



**Hill-Climbing
Search**

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ *problem*.INITIAL

while *true* **do**

neighbor $\leftarrow$ a highest-valued successor state of *current*

if VALUE(*neighbor*) $\leq$ VALUE(*current*) **then return** *current*

current $\leftarrow$ *neighbor*

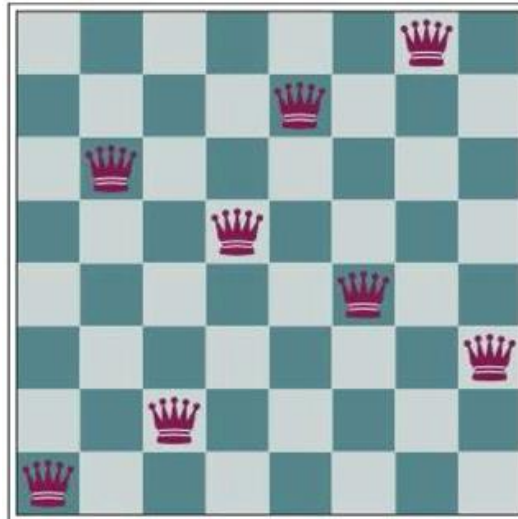
- Hill climbing is sometimes called **greedy local search** because it grabs a good neighbor state without thinking ahead about where to go next.

Search in
Complex
Environments

Hill-Climbing
Search

8 Queens Problem

- To illustrate hill climbing, we will use the 8-queens problem, where each state has 8 queens on the board, one per column.
- The successors of a state are all possible states generated by moving a single queen to another square in the same column (so each state has $8 \times 7 = 56$ successors).
- The heuristic cost function is the number of pairs of queens that are attacking each other, either directly or indirectly.
- The global minimum of this function is zero, which occurs only at perfect solutions.



$h = 1$

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	13	16	13	16	
14	17	15	14	16	16		
17	16	18	15	14	15		
18	14	15	15	14	16		
14	14	13	17	14	12	18	

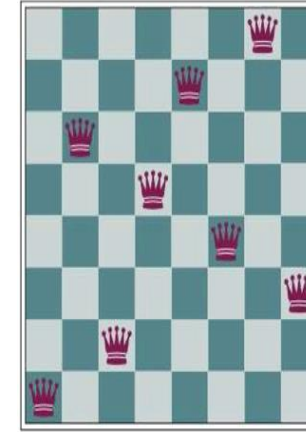
$h = 17$

Search in
Complex
Environments

**Hill-Climbing
Search**

Solving 8 Queens Problem Using Hill-Climbing Search

- Hill climbing often makes **rapid progress** toward a solution because it is usually quite easy to improve a bad state. For example, from the state in Figure 4.3(a), it takes just **five steps** to reach the state in Figure 4.3(b), which has $h=1$ and is very nearly a solution.
- Unfortunately, hill climbing often gets stuck for the following reasons:
 - Local maxima**: a local maximum is a peak that is higher than each of its neighboring states but lower than the global maximum.
 - Ridges**: Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.
 - Plateaux**: a plateau is a flat area of the state-space landscape. It can be a flat local maximum, from which no uphill exit exists, or a shoulder, from which progress is possible. A hill-climbing search might get lost on the plateau.

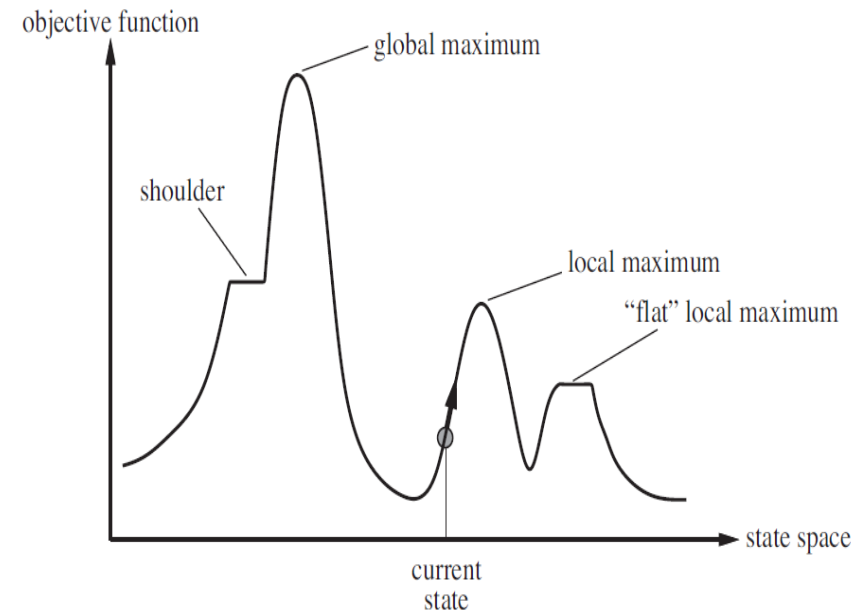
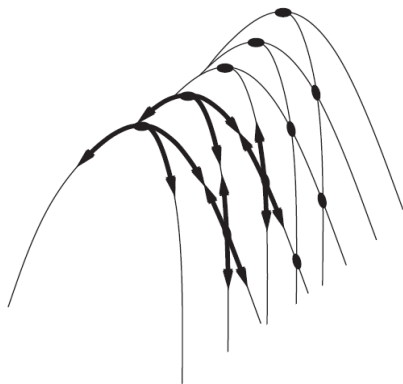


(a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	13	16	13	16	16
17	14	17	15	14	16	16	16
17	16	18	15	14	15	16	16
18	14	15	15	14	16	16	16
14	14	13	17	12	14	12	18

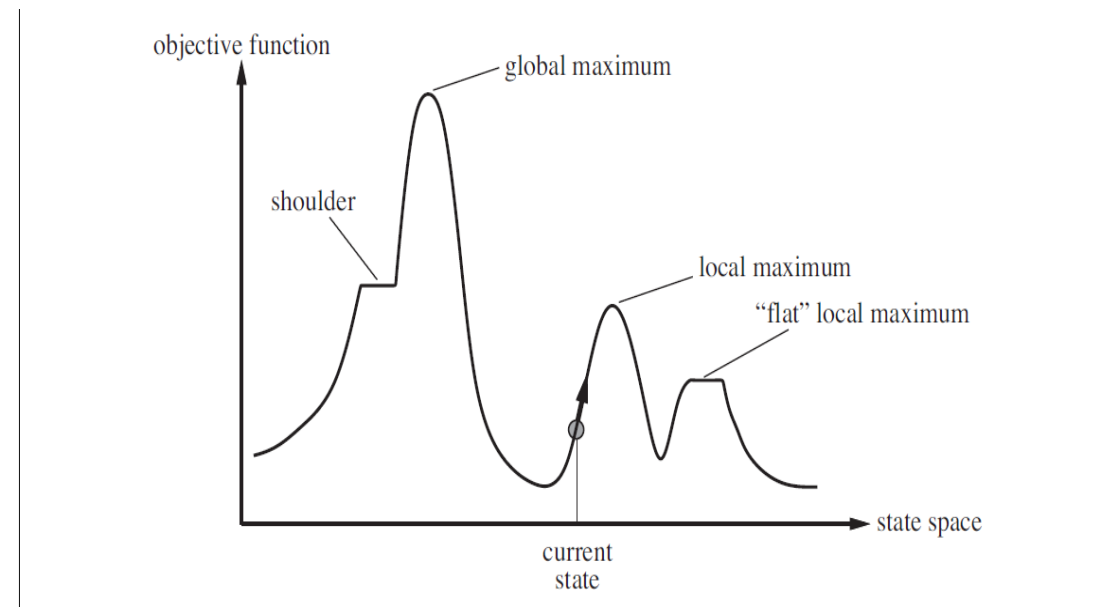
(b)

Search in
Complex
Environments



Hill-Climbing
Search

- In each case, the algorithm reaches a point at which no progress is being made. Starting from a randomly generated 8-queens state, steepest-ascent hill climbing gets stuck **86%** of the time, solving only **14%** of problem instances.
- It works quickly, taking just 4 steps on average when it succeeds and 3 when it gets not bad for a state space with $8^8 \approx 17$ million states.
- If we always allow sideways moves when there are no uphill moves, an infinite loop will occur whenever the algorithm reaches a flat local maximum that is not a shoulder.
- One common solution is to put a limit on the number of consecutive sideways moves allowed. For example, we could allow up to, say, 100 consecutive sideways moves in the 8-queens problem. This raises the percentage of problem instances solved by hill climbing from 14% to 94%. Success comes at a cost: the algorithm averages roughly 21 steps for each successful instance and 64 for each failure.



- Many variants of hill climbing have been invented. **Stochastic hill climbing** chooses at random from among the uphill moves; the probability of selection can vary with the steepness of the uphill move. This usually converges more slowly than steepest ascent, but in some state landscapes, it finds better solutions.
- **First-choice hill climbing** implements stochastic hill climbing by generating successors randomly until one is generated that is better than the current state. This is a good strategy when a state has many (e.g., thousands) of successors.
- The hill-climbing algorithms described so far are they often fail to find a goal when one exists because they can get stuck on local maxima. **Random-restart hill climbing** adopts the well-known adage, “**If at first you don’t succeed, try, try again**” It conducts a series of hill-climbing searches from randomly generated initial states, until a goal is found. It is trivially complete with probability approaching 1.
- For 8-queens, then, random-restart hill climbing is very effective indeed. Even for three million queens, the approach can find solutions in under a minute.

Solving 8 Queens using Random Restart Hill Climbing

Search in
Complex
Environments

Hill-Climbing
Search